

REST API Security Overview

 Lecture Notes

Topic: REST API Security – CORS, XSS, DDoS | Authentication & Authorization | Session Handling in REST

Duration: 2 Hours
Lecture Type: Theory
Designed for: Computer Science Students

 Lecture Plan Overview (2 Hours)

Time	Topic
0–30 min	Introduction to REST API Security + CORS
30–60 min	XSS & DDoS
60–90 min	Authentication vs Authorization
90–120 min	Session Handling in REST + Wrap-up Activity

 0–30 Minutes

 Introduction to REST API Security

♦ **What is REST?**

REST (Representational State Transfer) is an architectural style for designing web services. REST APIs use HTTP methods:

- GET
- POST
- PUT

- DELETE

Real-Life Example:

Think of a **restaurant**:

- Menu = API endpoints
 - Order = Request
 - Food served = Response
 - Waiter = Server
-



Why Security is Important?

If APIs are not secured:

- Hackers can steal user data
- Attackers can overload servers
- Unauthorized users can access restricted data

Example:

Imagine a banking API without authentication — anyone could check or transfer money!



CORS (Cross-Origin Resource Sharing)

Definition:

CORS is a security mechanism that controls how resources on a web server can be requested from another domain.

Browsers enforce CORS.

Real-Life Example:

You live in Hostel A. Only students from Hostel A can enter.

If someone from Hostel B wants to enter, permission is required.



Same-Origin Policy

By default, browsers block:

Frontend: <http://example.com>

API: <http://api.example.com>

Even though related, they are different origins.



How CORS Works

Server sends headers like:

Access-Control-Allow-Origin: `https://example.com`
Access-Control-Allow-Methods: `GET, POST`
Access-Control-Allow-Headers: `Content-Type`

CORS Security Best Practices

-  Never use `Access-Control-Allow-Origin: *` for sensitive APIs
 -  Allow only trusted domains
 -  Restrict methods
 -  Validate tokens server-side
-

Student Activity (5 Minutes)

Ask students:

1. Why does CORS exist?
 2. What happens if we allow `*` for banking APIs?
 3. Is CORS a server-side or browser-side security feature?
-

30–60 Minutes

XSS (Cross-Site Scripting)

Definition:

XSS is an attack where malicious scripts are injected into trusted websites.

Types of XSS

1. Stored XSS
 2. Reflected XSS
 3. DOM-based XSS
-

Real-Life Example:

A comment box allows users to post comments.

Attacker enters:

```
<script>alert("Hacked!")</script>
```

If not sanitized, every user visiting the page sees it.

Impact of XSS

- Cookie theft
 - Session hijacking
 - Redirect to malicious sites
-

Prevention Techniques

- Input validation
 - Output encoding
 - Use HTTPOnly cookies
 - Use Content Security Policy (CSP)
-

4 DDoS (Distributed Denial of Service)

Definition:

DDoS is an attack where multiple systems flood a server with traffic to make it unavailable.

Real-Life Example:

Imagine 10,000 fake customers enter a shop and don't buy anything — real customers cannot enter.

Famous Example:

In 2016, the company Dyn was attacked, affecting websites like:

- Twitter
- Netflix
- GitHub

(This is a real-world referenced event.)

DDoS Protection

- Rate limiting
- CAPTCHA
- API Gateway
- Load balancers
- CDN services

Student Quick Quiz (5 Minutes)

1. Difference between XSS and DDoS?
 2. Which attack targets users? Which targets servers?
 3. How does rate limiting help?
-

60–90 Minutes

Authentication vs Authorization

Authentication (AuthN)

Definition:

Verifying **who you are**.

Example:

- Username & password
 - OTP
 - Biometrics
-

Authorization (AuthZ)

Definition:

Verifying **what you can access**.

Example:

- Student cannot access admin panel
 - Teacher can upload marks
-

Real-Life Example

At University:

- ID Card Check = Authentication
 - Access to Lab = Authorization
-

Common Authentication Methods in REST

1. Basic Authentication
2. API Keys

3. Token-Based Authentication
 4. JWT (JSON Web Token)
 5. OAuth 2.0
-

JWT (JSON Web Token)

Definition:

JWT is a compact, URL-safe token format used for authentication.

Structure:

`Header.Payload.Signature`

Example Use Case:

User logs in → Server generates JWT → Client stores token → Sends token in header:

`Authorization: Bearer <token>`

OAuth 2.0

Used by:

- Google
- Facebook
- Microsoft

Example:

"Login with Google"

Class Discussion (5 Minutes)

Ask:

- Why is JWT preferred in REST?
 - Why not store passwords in frontend?
 - Is authentication enough without authorization?
-

90–120 Minutes

Session Handling in REST

What is a Session?

A session stores user state between requests.



Problem:

REST is **stateless**.

Each request must contain all information.



Two Approaches to Session Handling



Traditional Session-Based

- Server stores session
- Client stores session ID in cookie

Problem:

- Not scalable
 - Hard for microservices
-



Token-Based (Recommended for REST)

- Server issues JWT
- Client stores token
- Each request includes token

Advantages:

- Scalable
 - Stateless
 - Works with mobile apps
-



Secure Session Practices

- Use HTTPS
 - Set HttpOnly & Secure cookies
 - Short token expiry
 - Refresh tokens
 - Revoke compromised tokens
-



Complete Flow Example

1. User logs in
2. Server validates credentials

3. Server generates JWT
 4. Client stores JWT
 5. Client sends token in header
 6. Server verifies token
 7. Access granted or denied
-

Final Group Activity (10 Minutes)

Divide students into groups:

Design secure API for:

- Online Banking System
- E-Commerce Website
- University Portal

Each group must:

- Choose authentication method
- Explain how to prevent XSS
- Explain how to prevent DDoS
- Define authorization levels

Groups present for 2 minutes each.

Key Takeaways

- CORS protects browsers from cross-origin attacks
 - XSS targets users
 - DDoS targets availability
 - Authentication $\neq$ Authorization
 - REST prefers token-based session handling
 - Security must be multi-layered
-

Important Definitions (May Require Citation if Used in Research)

If students use definitions in assignments, they should cite:

- REST – defined by Roy Fielding in his doctoral dissertation (2000)
- OAuth 2.0 – IETF Standard

- JWT – RFC 7519
 - Same-Origin Policy – Web security model defined by browsers
-

End of Lecture Summary

Security in REST APIs is not optional — it is essential.

Without:

- Proper CORS → Browser misuse
- XSS protection → User data theft
- DDoS protection → Service unavailability
- AuthN & AuthZ → Unauthorized access
- Secure sessions → Account hijacking

A secure API is like a well-guarded university:

- Gatekeeper (Authentication)
- Access control (Authorization)
- CCTV & Rules (Security policies)
- Visitor log (Session handling)